

AgroSearch — Motor de Busca Inteligente

Relatório Técnico | AgroTech Solutions — Recuperação da Informação

Equipe	Disciplina / Turma	Data
[Aluno 1] • [Aluno 2] • [Aluno 3]	[Disciplina] — [Turma]	[_]/[_]/[_]

1. Objetivo e Escopo

Os técnicos de campo da AgroTech gastam tempo excessivo localizando informação em manuais internos de agricultura sustentável, controle de pragas e irrigação. O **AgroSearch** é um protótipo de motor de busca textual que recebe uma consulta em linguagem natural e devolve os documentos ordenados por relevância. Todo o núcleo — pré-processamento, índice invertido, TF-IDF e similaridade de cosseno — foi implementado **do zero**, em conformidade com a restrição técnica: não há `scikit-learn`, `NLTK`, `spaCy`, `gensim` ou `TfidfVectorizer`. Usamos apenas a biblioteca padrão do Python (`re`, `math`, `unicodedata`, `collections`); `pandas` e `Streamlit` atuam somente na camada de apresentação. A entrega é um único arquivo: `agrosearch.py`.

2. Arquitetura e Decisões de Implementação

Fase 1 — Pipeline de pré-processamento. Executado em quatro etapas encadeadas, todas inspecionáveis na interface: **(a) Normalização** — *case folding* e remoção de acentos por decomposição Unicode NFD, descartando os caracteres da categoria Mn (marcas diacríticas), seguida da eliminação de pontuação; **(b) Tokenização** — segmentação por expressão regular `[a-z0-9]+`, descartando tokens de um caractere; **(c) Stopwords** — lista própria de ~110 palavras funcionais do português, normalizada no mesmo espaço dos tokens para garantir o casamento; **(d) Stemming** — *stemmer* sufixal inspirado no RSLP (Orengo & Huyck, 2001), escrito do zero com sete blocos de regras aplicados em cascata (plural, feminino, advérbio, aumentativo/diminutivo, sufixos nominais, sufixos verbais e vogal temática). Cada regra exige um radical mínimo, evitando mutilar palavras curtas. Resultado prático: *irrigação*, *irrigar* e *irrigado* convergem para *irrig*, e *cultural/cultivo* para *cult* — o que eleva a revocação da busca.

Fase 2 — Índice invertido. Construído como `{termo: {doc_id: frequência}}`. Guardar a frequência já na *posting list* evita reprocessar os documentos no cálculo do TF. A estrutura troca a varredura linear do corpus por acesso direto em tempo médio $O(1)$ ao dicionário.

Ingestão de documentos. Além da base interna em texto, a aplicação aceita o **upload dos manuais em PDF**: o texto é extraído página a página, normalizado quanto a espaços e submetido ao mesmo pipeline. Ler o PDF é apenas desempacotar um formato binário — nenhuma etapa de RI é terceirizada. A aba de pré-processamento exibe um relatório de extração (páginas, caracteres, prévia e situação por arquivo), sinalizando PDFs digitalizados que exigiriam OCR.

Fase 3 — Ranqueamento. A consulta passa pelo *mesmo* pipeline dos documentos (requisito de consistência do espaço de indexação), é vetorizada e confrontada com a matriz TF-IDF. A interface expõe a memória de cálculo termo a termo.

3. Formulação Matemática

Métrica	Fórmula implementada	Interpretação
TF	$tf(t,d) = f(t,d) / d $	Peso local: importância do termo dentro do documento, normalizada pelo tamanho.
IDF	$idf(t) = \log_{10}(N / df_t)$	Peso global: raridade do termo no corpus. Se $df = N$, o termo não discrimina e o IDF zera.
TF-IDF	$w(t,d) = tf(t,d) \times idf(t)$	Combina relevância local e poder discriminativo global.
Score	$score(q,d) = \sum_{t \in q} w(t,d)$	Soma acumulada dos pesos dos termos da consulta; base do ranking.
Cosseno (<i>bônus</i>)	$cos(q,d) = (q \cdot d) / (q d)$	Normaliza a magnitude dos vetores; trata melhor consultas com múltiplas palavras.

A aplicação permite alternar entre esquemas alternativos (TF bruto e logarítmico; IDF suavizado), o que torna visível o efeito de cada escolha sobre o ranking.

4. Validação — Consulta “irrigação da soja”

Consulta processada: `[irrig, soja]`. Com $N = 5$: $df(irrig) = 2 \rightarrow idf = \log_{10}(5/2) = 0,39794$; $df(soja) = 3 \rightarrow idf = \log_{10}(5/3) = 0,22185$.

Pos.	Doc	d	TF-IDF acumulado	Cosseno	Termos casados
1º	Doc 1	9	0,06887	0,23921	irrig, soja
2º	Doc 5	7	0,05685	0,20923	irrig
3º	Doc 2	7	0,03169	0,06636	soja
4º	Doc 4	8	0,02773	0,06446	soja
5º	Doc 3	7	0,00000	0,00000	—

O **Doc 1** vence por ser o único a conter os dois termos da consulta. O Doc 5 supera Doc 2 e Doc 4 porque *irrig* é mais raro (IDF maior) que *soj* e porque seu documento é mais curto, elevando o TF normalizado. O Doc 3 é corretamente descartado (score nulo). Verificamos ainda os casos-limite: consulta vazia, consulta só com stopwords, termo fora do vocabulário e corpus editado pelo usuário. Os mesmos cinco documentos, convertidos em PDF e reenviados pela interface, reproduzem exatamente os valores acima — a extração não altera o corpus.

5. Interface (Streamlit)

A aplicação unifica os três pilares em quatro abas — *Pré-processamento*, *Índice Invertido*, *Busca & Ranqueamento* e *Fundamentos* — com as opções de *Stopwords* e *Stemming* em *checkboxes* na barra lateral. Ao alterná-las, o vocabulário, o índice e o ranking se recalculam em tempo real, evidenciando o impacto de cada etapa (33 termos com o pipeline completo contra 44 sem stopwords, no corpus de 5 documentos). A aba 1 mostra a transformação passo a passo de cada documento; a aba 2 exibe o índice em `st.dataframe/st.json` e a matriz TF-IDF como mapa de calor; a aba 3 apresenta o ranking ordenado, destaca o documento vencedor, realça os termos casados no trecho recuperado e abre a memória de cálculo. O corpus é editável pela barra lateral, permitindo testes com novos manuais.

6. Divisão de Tarefas

Integrante	Responsabilidade principal	Entregas
[Aluno 1]	Fase 1 — Pipeline de pré-processamento	Normalização Unicode, tokenizador, lista de stopwords, stemmer RSLP simplificado e testes de radicais.
[Aluno 2]	Fase 2 e 3 — Índice invertido e ranqueamento	Construção do índice, funções de TF/IDF/TF-IDF, similaridade de cosseno e validação numérica manual.
[Aluno 3]	Interface, integração e documentação	Layout Streamlit, abas, memória de cálculo, testes de casos-limite e redação do relatório.

Revisão cruzada de código entre os integrantes; versionamento em repositório Git com commits por fase.

7. Limitações e Trabalhos Futuros

- O *stemmer* é heurístico e pode gerar *overstemming* (radicais distintos que colapsam) ou *understemming*; um lematizador com dicionário reduziria o erro.
- O modelo é *bag-of-words*: ignora ordem, negação e sinonímia — “gotejamento” e “irrigação localizada” não se conectam. Buscas por sinônimos exigiriam expansão de consulta ou *embeddings*.
- O índice reside em memória, adequado ao protótipo; escalar para milhares de manuais exigiria persistência, compressão das *posting lists* e indexação incremental.
- Evoluções previstas: BM25 (saturação de TF e normalização por tamanho), busca por frase com índice posicional e destaque de trechos (*snippets*) longos.

8. Rastreabilidade dos Critérios

Critério	Onde está implementado (agrosearch.py)	Como verificar na aplicação
Pipeline	<code>normalizar()</code> , <code>tokenizar()</code> , <code>remover_stopwords()</code> , <code>stemizar()</code> — seção 2	Aba 1: transformação passo a passo e tabela token → stem.
Índice invertido	<code>construir_indice_invertido()</code> — seção 3	Aba 2: tabela com DF e posting list, ou visão em <code>st.json</code> .
TF-IDF	<code>calcular_tf()</code> , <code>calcular_idf()</code> , <code>construir_matriz_tfidf()</code> , <code>buscar()</code> — seção 4	Aba 3: ranking ordenado e memória de cálculo termo a termo.
Interface	<code>main()</code> — seção 6	Quatro abas, checkboxes de stopwords/stemming e corpus editável.
Ingestão de PDFs	<code>extrair_texto_pdf()</code> , <code>carregar_arquivos()</code> — seção 2.5	Barra lateral: “Upload de PDFs”; relatório de extração na aba 1.
Bônus — cosseno	<code>buscar()</code> : vetor da consulta, produto escalar e normas	Aba 3: alternar a ordenação para “Similaridade de Cosseno”.